DOCUMENTATION DU PROJET

CACHE-CACHE

EN LIGNE

Principe du jeu, éléments, fonctionnement et techniques employées

Jeu de cache-cache multijoueur en pixel art — Node.js, Socket.IO et Canvas

Version du 7 novembre 2026

Code source : github.com/animaappcontact-stack/cache-cache-game

Sommaire

1. Présentation générale 4
1.1 Le concept 4
1.2 Objectifs du projet 4
1.3 Chiffres clés 4
1.4 Technologies utilisées 4
2. Principe du jeu et règles 6
2.1 Les rôles 6
2.2 Déroulement d'une partie 6
2.3 Conditions de victoire 6
2.4 Les mécaniques de jeu 6
3. Commandes et interface 8
3.1 Commandes 8
3.2 Les écrans 8
4. Les éléments du jeu 10
4.1 Les personnages 10
4.2 Les cachettes 10
4.3 Le mobilier et le décor 10
4.4 Murs, sols et escaliers 11
4.5 Les cinq cartes 11
5. Architecture technique 14
5.1 Vue d'ensemble 14
5.2 Structure des fichiers 14
5.3 Rôle de chaque fichier 14
5.4 Stockage des données 14
5.5 Dépendances 15
6. Fonctionnement réseau 16
6.1 Le modèle « hôte autoritaire » 16
6.2 Cycle réseau d'une partie 16
6.3 Les événements Socket.IO 16
6.4 Gestion des salons 17
6.5 Contenu d'un état synchronisé 17
6.6 Réduction du lag 17
6.7 Prédiction et réconciliation (invités) 18
7. Le moteur de jeu 19
7.1 La boucle de jeu 19
7.2 La grille et les coordonnées 19

7.3 Déplacements et collisions 19
7.4 La ligne de vue et le brouillard de guerre 19
7.5 La caméra et l'ordre de dessin 20
7.6 Animation des sprites 20
7.7 Les cachettes : une machine à états 20
7.8 Le dash 20
7.9 La minimap 21
7.10 Le temps et la fin de partie 21
8. Conception des cartes 22
8.1 Représentation 22
8.2 Les outils de construction 22
8.3 Taille et rotation des meubles 22
8.4 Calage automatique aux murs 22
8.5 Cartes de tailles différentes 23
8.6 Vérification des cartes 23
8.7 Ajouter une nouvelle carte 23
9. Interface et style 24
9.1 Gestion des écrans 24
9.2 Identité visuelle 24
9.3 Adaptation aux petits écrans 24
9.4 Le bouton de lancement 24
10. Déploiement 25
10.1 Le code sur GitHub 25
10.2 L'hébergement sur Render 25
10.3 Lancer le jeu en local 25
10.4 Limites du plan gratuit et bonnes pratiques 25
11. Sécurité, qualité et tests 26
11.1 Règles contrôlées par le serveur 26
11.2 Dépendances et vulnérabilités 26
11.3 Méthodes de test employées 26
11.4 Limites connues 26
12. Historique des évolutions 27
13. Pistes d'amélioration 28
Annexe A — Glossaire 29
Annexe B — Paramètres du jeu 30
Annexe C — Liste des types de meubles 31

Si le sommaire apparaît vide, clic droit dessus puis « Mettre à jour les champs » dans Word.

1. Présentation générale

1.1 Le concept

Cache-Cache en ligne est un jeu multijoueur en temps réel, jouable directement dans un navigateur web, sans installation. Il reprend le principe du cache-cache traditionnel avec une ambiance de film d'horreur en pixel art : un **Chasseur** parcourt une maison pour retrouver des **Traqués** qui se sont dissimulés dans les meubles.

La partie se joue en vue de dessus. Chaque joueur ne voit qu'une petite zone autour de lui, le reste de la carte étant plongé dans le noir. Les traqués doivent se glisser dans une cachette (lit, armoire, douche, voiture…) avant d'être repérés ; le chasseur doit fouiller les meubles un par un ou surprendre ses proies à découvert.

1.2 Objectifs du projet

- Proposer un jeu **jouable à plusieurs à distance** (de 2 à 5 joueurs + spectateurs), accessible par un simple lien.
- Mettre en œuvre une **architecture client / serveur temps réel** avec WebSocket (Socket.IO).
- Concevoir un **moteur de jeu 2D** complet en JavaScript natif : collisions, ligne de vue, brouillard de guerre, caméra, animations.
- Fournir plusieurs **cartes** de tailles et d'ambiances différentes, faciles à étendre.
- Déployer le jeu en ligne (GitHub + Render) et le rendre **fluide** même pour les joueurs distants.

1.3 Chiffres clés

Élément	Valeur
Code serveur (`server.js`)	environ 290 lignes
Code client (`public/client.js`)	environ 2 090 lignes
Interface (`index.html` + `style.css`)	environ 830 lignes
Cartes jouables	5 (dont 2 grandes cartes de 12 et 15 pièces)
Images (sprites, meubles, titre)	62 fichiers dans `public/assets/`
Joueurs par salon	minimum 2 ; 1 chasseur maximum ; 4 traqués maximum ; spectateurs illimités
Durées de partie	2, 3 ou 4 minutes (au choix de l'hôte)
Dépendances npm	2 : `express` et `socket.io`

1.4 Technologies utilisées

Domaine	Technologie	Utilisation
Serveur	Node.js + Express 5	Sert les fichiers du jeu et héberge le serveur de salons.
Temps réel	Socket.IO 4 (WebSocket)	Communication bidirectionnelle entre les joueurs et le serveur.

Domaine	Technologie	Utilisation
Rendu	HTML5 Canvas 2D	Dessin de la carte, des personnages, du brouillard et de la minimap.
Logique	JavaScript natif (sans framework)	Moteur de jeu, physique, réseau côté client.
Interface	HTML5 + CSS3	Écrans (accueil, salon, partie), police pixel « Press Start 2P ».
Versionnage	Git + GitHub	Historique et sauvegarde du code.
Hébergement	Render (Web Service)	Mise en ligne automatique à chaque envoi sur GitHub.

2. Principe du jeu et règles

2.1 Les rôles

Chaque joueur choisit un rôle dans le salon avant le lancement. Un joueur qui n'a pas choisi reste **spectateur** : il n'apparaît pas dans la partie.

Rôle	Effectif	Capacités et contraintes
Chasseur	1 maximum	Immobilisé pendant les 10 premières secondes. Vision étendue (250 px). Fouille les meubles (touches E ou F). Élimine un traqué visible en le touchant.
Traqué	4 maximum	Petit personnage, vision réduite (150 px). Peut se cacher dans certains meubles (touche E) et utiliser un **dash** (Maj) pour fuir.
Spectateur	illimité	Rôle par défaut en arrivant dans le salon. Ne joue pas et n'est pas dessiné.

2.2 Déroulement d'une partie

1. **Accueil** : le joueur saisit un pseudo. Il laisse le code vide pour créer un salon, ou saisit le code à 6 caractères d'un salon existant pour le rejoindre.
2. **Salon** : chacun choisit son rôle ; l'hôte (premier arrivé, couronne affichée) choisit la carte et la durée de la partie.
3. **Lancement** : seul l'hôte peut lancer la partie, et seulement s'il y a **au moins 2 joueurs** dans le salon. Le bouton reste grisé tant que ce n'est pas le cas.
4. **Phase de cachette** : le chasseur voit un compte à rebours noir de 10 secondes pendant lequel il ne peut pas bouger. Les traqués en profitent pour se cacher.
5. **Chasse** : le chasseur fouille la maison. Chaque traqué trouvé ou touché est éliminé.
6. **Fin de partie** : un message de victoire s'affiche ; chacun peut revenir au salon pour rejouer.

2.3 Conditions de victoire

Camp	Condition
Chasseur	Tous les traqués de la partie sont éliminés (il y avait au moins un traqué).
Traqués	Le temps est écoulé et au moins un traqué est encore en vie.

2.4 Les mécaniques de jeu

Se cacher

Un traqué proche d'une cachette (à moins de 30 pixels de son bord) appuie sur **E** : il est téléporté au centre du meuble, qui se ferme derrière lui, et devient invisible pour tout le monde sauf pour lui-même. Un nouvel appui sur **E** le fait ressortir à l'endroit exact où il était entré.

Fouiller

Le chasseur proche d'un meuble appuie sur **E** ou **F** : le meuble s'ouvre. Si un traqué est caché dedans, il est éliminé et le meuble prend une apparence spéciale (image « occupée »). Un meuble ouvert reste ouvert et devient traversable.

Élimination au contact

Un traqué qui n'est pas caché est éliminé dès que le chasseur le touche (distance inférieure à la taille d'un personnage, soit 24 pixels). Son corps reste visible au sol.

Le dash

Réservé aux traqués. Appuyer sur **Maj** en se déplaçant déclenche une accélération de **0,6 seconde** à **2,2 fois** la vitesse normale. Le dash se **recharge 8 secondes après sa fin**. Il est impossible à l'arrêt ou caché. Un halo bleu marque le traqué en plein dash et un indicateur du HUD affiche son état.

Le brouillard de guerre

Seule la zone proche du joueur est visible, et uniquement ce qui est en **ligne de vue** (les murs bloquent la vue). La vision est plus large pour le chasseur (250 px) que pour les traqués (150 px). Au-delà, l'écran est noir.

La minimap

La minimap affiche les murs et les meubles de la carte mais **uniquement la position du joueur lui-même**. Voir l'autre camp sur la minimap supprimerait tout l'intérêt de se cacher.

> **Équilibrage —** le chasseur voit plus loin mais commence immobile ; les traqués voient peu mais disposent de cachettes et d'un dash. Les paramètres sont regroupés dans l'annexe B.

3. Commandes et interface

3.1 Commandes

Les commandes sont pensées pour un clavier AZERTY. Les flèches directionnelles fonctionnent aussi.

Touche	Action
Z ou flèche haut	Aller vers le haut
S ou flèche bas	Aller vers le bas
Q ou flèche gauche	Aller vers la gauche
D ou flèche droite	Aller vers la droite
E	Traqué : se cacher / sortir de sa cachette. Chasseur : fouiller le meuble proche.
F	Chasseur : fouiller le meuble proche (même effet que E).
Maj (Shift)	Traqué : dash dans la direction du déplacement.

Si la fenêtre perd le focus (alt-tab), toutes les touches sont automatiquement relâchées pour éviter qu'un personnage ne reste « coincé » en mouvement.

3.2 Les écrans

Accueil

Le titre « Cache Cache » est affiché en grand dans le fond de l'écran, sans être rogné. Une boîte centrale permet de saisir son pseudo (12 caractères maximum) et le code d'un salon (6 caractères).

Salon

Le salon affiche le code à communiquer aux amis, les trois listes de joueurs (chasseurs, traqués, spectateurs) avec la couronne de l'hôte, le choix du rôle, le choix de la carte, la durée (2, 3 ou 4 minutes) et le bouton de lancement. Seul l'hôte peut modifier la carte et la durée ; ces réglages sont grisés pour les autres.



Figure 1 — Écran d'accueil

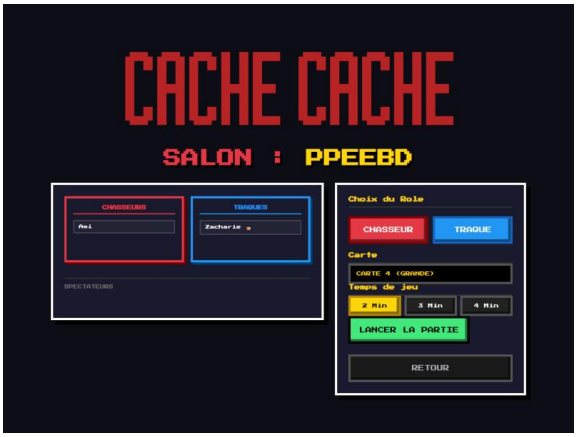


Figure 2 — Salon d'attente

Partie

L'écran de jeu contient le canvas principal (vue centrée sur le joueur), le HUD, la minimap et un bouton pour revenir au salon.

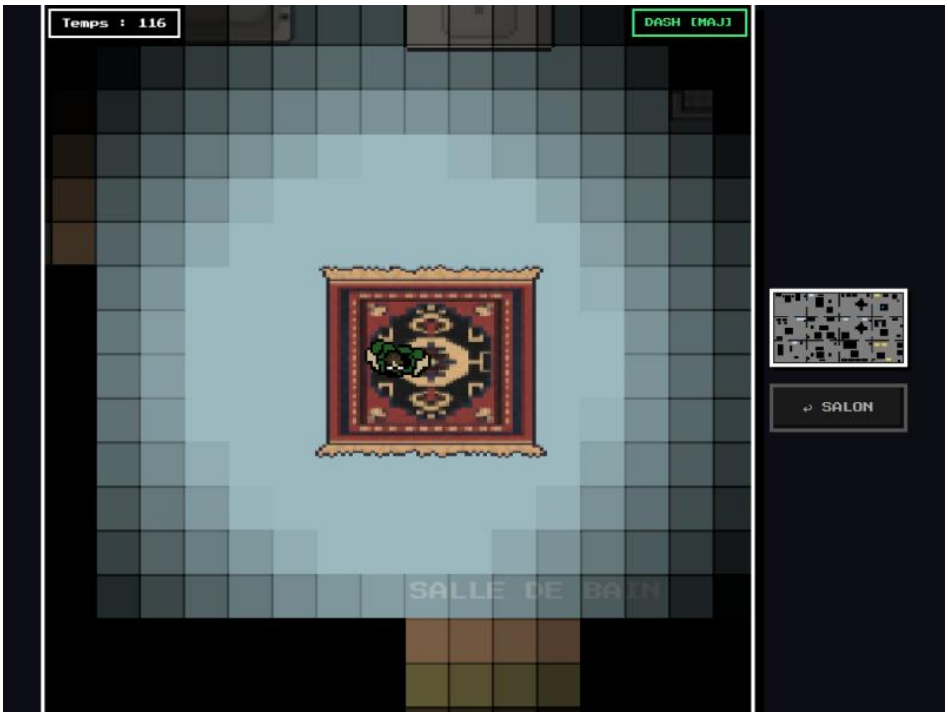


Figure 3 — Écran de jeu vu par un traqué (brouillard, HUD, minimap)

Élément du HUD	Contenu
En haut à gauche	Temps restant en secondes (« Temps : 116 »). Pour le chasseur au départ : « ATTENDS : 10s ».
En haut à droite (traqués)	« DASH [MAJ] » (prêt, vert), « DASH ! » (en cours, bleu) ou « DASH 5s » (rechargement, gris).
Au centre	Message de victoire en fin de partie.
Colonne de droite	Minimap (joueur local uniquement) et bouton « SALON ».
Au sol	Nom discret de la pièce dans laquelle on se trouve (grandes cartes).

Écrans hérités

Le code contient encore des écrans d'une version précédente : « mode film », « nouveau mode », « chargement » et « game over » avec défilement de crédits. Ils ne sont plus accessibles depuis le salon (leurs boutons ont été retirés) mais leur code subsiste.

4. Les éléments du jeu

4.1 Les personnages

Caractéristique	Chasseur	Traqué
Vitesse de déplacement	120 px/s	120 px/s (264 px/s pendant le dash)
Boîte de collision	24 × 24 px	24 × 24 px
Rayon de vision	250 px (zone nette jusqu'à 125 px)	150 px (zone nette jusqu'à 75 px)
Taille d'affichage	sprite agrandi ×1,4	sprite réduit ×0,5 (×1 une fois mort)
Sprites	immobile + 2 images de marche, 4 directions	1 image par direction + image « mort »

Les sprites de marche du chasseur alternent entre deux images toutes les 200 ms tant que le joueur se déplace. La direction affichée (haut, bas, gauche, droite) est celle du déplacement dominant.

4.2 Les cachettes

Huit types de meubles peuvent servir de cachette. Chacun existe en plusieurs états, chaque état ayant sa propre image : **fermé** (état normal), **ouvert** (fouillé ou quitté) et **occupé** (un traqué a été trouvé dedans).

Cachette	Particularités
Lit simple	Existe en trois états (fermé, ouvert, occupé).
Lit double	Idem, avec une image plus large.
Armoire	Trois états.
Dressing	Trois états ; placé dos au mur sur les grandes cartes.
Douche	Trois états.
Voiture (Kangoo)	Trois états.
Tapis	Reste **traversable** quel que soit son état ; peut cacher un traqué.
Baignoire	Pas d'image « occupé » : le corps tombe au sol à côté.

4.3 Le mobilier et le décor

Les autres meubles sont **solides** : ils bloquent les déplacements mais ne servent pas de cachette. Ils structurent les pièces et créent des obstacles qui coupent la ligne de vue.

Catégorie	Éléments
Salon / séjour	canapé, fauteuil, télévision, table basse et tables, chaises, étagères, plantes, lampe
Chambre	tables de chevet, bureau avec ordinateur, chaise de bureau
Cuisine	réfrigérateur, plaques de cuisson, lavabo, table et chaises
Salle de bain	lavabo, toilettes (WC)

Catégorie	Éléments
Garage / atelier	moto, malle, chaudière
Loisirs	table de billard

4.4 Murs, sols et escaliers

- **Murs** : dessinés sur une grille de tuiles de 16 px. Ils bloquent à la fois les déplacements et la vue.
- **Sol** : une couleur beige uniforme sur les cartes 1 à 3 ; sur les grandes cartes, chaque pièce a une couleur propre (bleu pour les chambres, vert feutré pour la salle de jeux, gris pour le garage…) et son nom écrit au sol.
- **Escaliers** : purement visuels (dégradé de marches) sur les cartes 1 et 3. Ils servent de repère et ne changent pas d'étage.

4.5 Les cinq cartes

N°	Nom	Taille	Description
1	Carte 1 (maison)	56 × 30 tuiles (896 × 480 px)	Maison aux grandes pièces ouvertes : chambre avec lit double, bureau, alcôves avec dressing et escalier central.
2	Carte 2	56 × 30 tuiles	Logement de plain-pied : séjour, salle à manger, cuisine, chambres, salle d'eau et garage.
3	Carte 3	56 × 30 tuiles	Maison à escalier central avec salon, chambres, cuisine et salles de bain.
4	Carte 4 — le Manoir	88 × 48 tuiles (1 408 × 768 px)	12 pièces différentes en grille 4 × 3, reliées par des portes de 4 tuiles.
5	Carte 5 — la Villa	104 × 56 tuiles (1 664 × 896 px)	Grand salon central à piliers entouré de 14 pièces différentes.



Figure 4 — Carte 1 : la maison (vue d'ensemble sans brouillard)



Figure 5 — Carte 2



Figure 6 — Carte 3 (escalier central)



Figure 7 — Carte 4 : le Manoir (12 pièces)



Figure 8 — Carte 5 : la Villa (grand salon central et 14 pièces)

Les pièces du Manoir

Chambre parentale, salon, cuisine, salle de bain, bureau, salle de jeux (billard), salle à manger, chambre d'enfants, garage, cinéma maison, dressing et spa.

Les pièces de la Villa

Grand salon central, bibliothèque, chambre, cuisine, salon, salle de bain, bureau, chambre d'enfants, atelier, jardin d'hiver, garage, dressing, salle de jeux, buanderie et chambre d'ado.

> **À noter —** sur les grandes cartes, plantes, canapés et dressings sont automatiquement collés aux murs pour un rendu réaliste (voir 8.4), et chaque pièce a un thème, des meubles et une couleur uniques.

5. Architecture technique

5.1 Vue d'ensemble

Le projet suit une architecture **client / serveur**. Le serveur Node.js ne contient quasiment aucune logique de jeu : il gère les salons et **relaie les messages** entre les joueurs. Toute la simulation (déplacements, collisions, éliminations) est calculée par le navigateur de l'**hôte** du salon.

Navigateur invité	Serveur Node.js (Render)	Navigateur hôte
Envoie ses touches Prédit son propre mouvement Affiche l'état reçu	⇄ Socket.IO ⇄ Gère les salons et les rôles Relaie les touches et l'état	Reçoit les touches de tous **Calcule la simulation** Diffuse l'état 20 fois par seconde

5.2 Structure des fichiers

```
Cache-cache Game/
├── server.js            Serveur : salons, rôles, relais temps réel
├── package.json         Dépendances (express, socket.io) et script start
├── package-lock.json    Versions exactes installées
├── .gitignore           Exclut node_modules
└── public/              Tout ce qui est envoyé au navigateur
    ├── index.html       Structure des écrans
    ├── style.css        Mise en forme (thème pixel art)
    ├── client.js        Moteur de jeu, réseau client, rendu
    └── assets/          62 images (sprites, meubles, titre)
```

5.3 Rôle de chaque fichier

Fichier	Contenu et responsabilités
`server.js`	Crée le serveur HTTP Express, sert le dossier `public`, gère les salons en mémoire, applique les règles du salon (rôles, hôte, durée, carte, minimum de joueurs), relaie les touches et l'état de jeu, nettoie les salons inactifs.
`public/index.html`	Contient tous les écrans (accueil, salon, partie, écrans hérités) ; seul l'écran actif est visible, les autres portent la classe `hidden`.
`public/style.css`	Thème sombre, police pixel, boutons, panneaux, mise en page responsive, HUD, titre en fond d'accueil et bannière du salon.
`public/client.js`	Tout le côté client : chargement des images, commandes clavier, interface du salon, construction des cartes, boucle de jeu, physique, ligne de vue, brouillard, dessin, minimap, réseau, prédiction.

5.4 Stockage des données

Il n'y a **aucune base de données**. L'état de tous les salons est conservé dans un simple objet JavaScript en mémoire côté serveur. Conséquence : un redémarrage du serveur efface tous les salons en cours, ce qui est acceptable pour un jeu de parties courtes.

5.5 Dépendances

Paquet	Version	Rôle
`express`	^5.2.1	Serveur web : distribue `index.html`, `style.css`, `client.js` et les images.
`socket.io`	^4.8.3	Communication temps réel ; fournit aussi la bibliothèque client `/socket.io/socket.io.js`.

6. Fonctionnement réseau

6.1 Le modèle « hôte autoritaire »

Dans chaque salon, le **premier joueur arrivé est l'hôte**. Son navigateur fait tourner la simulation et fait autorité sur l'état du jeu. Les autres joueurs (invités) envoient seulement leurs touches et affichent ce que l'hôte leur renvoie.

Avantages	Inconvénients
Aucun serveur de jeu à développer ni à payer : le serveur reste très léger. Un seul endroit où la vérité du jeu est calculée : pas de conflit.	L'hôte a un avantage de latence (il joue sans délai réseau). Un hôte malveillant pourrait tricher. Si l'hôte quitte, la partie dépend d'une migration d'hôte.

6.2 Cycle réseau d'une partie

1. Le joueur envoie `createRoom` ou `joinRoom` ; le serveur répond `lobbyJoined` puis diffuse `playersUpdated` à tout le salon.
2. Les joueurs choisissent leur rôle et l'hôte règle carte et durée via `lobbyAction`.
3. L'hôte envoie `startGame`. Le serveur vérifie qu'il est bien hôte et qu'il y a au moins 2 joueurs, lance le chrono et diffuse `gameStarted`.
4. Chaque navigateur construit la carte localement (la carte est déterminée par son numéro, rien n'est transféré).
5. Pendant la partie : les invités envoient `playerInput` ; le serveur le relaie à l'hôte sous le nom `clientInput` ; l'hôte calcule et envoie `stateSnapshot` ; le serveur diffuse `syncState` aux invités.
6. À la fin, l'hôte renvoie tout le monde au salon avec `returnToLobby`.

6.3 Les événements Socket.IO

Événement	Sens	Rôle
`createRoom`	client → serveur	Crée un salon (code à 6 caractères) ; le créateur devient hôte.
`joinRoom`	client → serveur	Rejoint un salon. Refusé si le salon n'existe pas ou si la partie a commencé.
`lobbyJoined`	serveur → client	Confirme l'entrée : code, statut d'hôte, réglages.
`playersUpdated`	serveur → salon	Liste des joueurs et de leurs rôles.
`lobbyAction`	client → serveur	Choix du rôle ; réglages de durée (`SET_TIME_*`) et de carte (`SET_MAP_*`), réservés à l'hôte.
`settingsUpdated`	serveur → salon	Nouveaux réglages (durée, carte).
`startGame`	client → serveur	Lancement par l'hôte (minimum 2 joueurs).
`gameStarted`	serveur → salon	Passage à l'écran de jeu.

Événement	Sens	Rôle
`playerInput`	invité → serveur	État des touches de l'invité.
`clientInput`	serveur → hôte	Touches d'un invité, avec son identifiant.
`stateSnapshot`	hôte → serveur	État compact de la partie, envoyé par l'hôte seul.
`syncState`	serveur → invités	Relais de l'état (envoi « volatile », voir 6.6).
`timerUpdate`	serveur → salon	Temps restant, chaque seconde.
`returnToLobby` / `returnedToLobby`	client ↔ serveur	Retour au salon (tout le monde si c'est l'hôte).
`leaveRoom`	client → serveur	Quitte le salon.
`hostMigrated`	serveur → client	Prévient le joueur devenu hôte.
`errorMsg`	serveur → client	Message d'erreur affiché à l'écran.

6.4 Gestion des salons

- **Code du salon** : 6 caractères alphanumériques aléatoires, vérifiés uniques.
- **Rôles** : un chasseur au maximum, quatre traqués au maximum ; les demandes au-delà sont ignorées.
- **Hôte** : le premier client de la liste du salon. S'il part, le joueur suivant devient hôte (`hostMigrated`).
- **Salon vide** : supprimé aussitôt.
- **Nettoyage** : toutes les 10 minutes, un « éboueur » supprime les salons inactifs depuis plus d'une heure.
- **Sécurité de jeu** : on ne peut pas rejoindre un salon dont la partie est en cours.

6.5 Contenu d'un état synchronisé

Chaque message d'état (`syncState`) contient, pour chaque joueur, uniquement les champs utiles à l'affichage :

- identifiant, pseudo, rôle ;
- position (x, y) arrondie au dixième de pixel, taille, vitesse ;
- vivant, caché, direction, en mouvement, minuterie d'animation ;
- minuteries du dash (durée restante, rechargement).

S'y ajoutent le temps restant, le compte à rebours du chasseur et, quand nécessaire, la liste compacte des états des meubles (ouvert/fermé/occupé). Un message pèse en moyenne **environ 460 octets**.

6.6 Réduction du lag

Lors d'un essai à quatre joueurs sur Render, les invités subissaient de fortes saccades. Une analyse du flux réseau a mené à cinq corrections :

Problème constaté	Solution mise en place
L'hôte envoyait l'état à **chaque image** (60 fois par seconde ou plus), avec **tous les meubles** : des dizaines de Ko par message.	Envoi **limité à 20 fois par seconde** sous forme compacte ; les meubles ne partent que s'ils changent (plus un rappel toutes les 2 s).
Chaque invité attendait l'aller-retour complet (touche → serveur → hôte → serveur → écran) avant de voir son personnage bouger.	**Prédiction locale** : l'invité déplace tout de suite son personnage avec les mêmes règles que l'hôte ; l'hôte le recale ensuite si besoin.
Les autres joueurs « sautaient » d'une position à l'autre à chaque message.	**Interpolation** : leur affichage glisse en douceur vers la dernière position reçue.
Les touches étaient envoyées 30 fois par seconde, même sans changement.	Envoi **uniquement quand une touche change**, plus un battement de secours toutes les 500 ms.
Les messages en retard s'empilaient et le décalage grandissait.	Envois **« volatiles »** : un état périmé est abandonné au lieu d'être mis en file ; connexion **WebSocket directe** sans long-polling.

> **Résultat mesuré —** environ 7 Ko/s de trafic d'état pour 2 joueurs en local, contre plusieurs Mo/s auparavant.

6.7 Prédiction et réconciliation (invités)

Pour que le personnage de l'invité réagisse instantanément, le client applique **les mêmes fonctions** que l'hôte (`applyMovement` et `collides`). À chaque état reçu de l'hôte :

- si l'écart entre la position prédite et la position officielle dépasse **70 px** (téléportation, cachette, mort…), le joueur est **recalé d'un coup** ;
- si le joueur est **à l'arrêt**, sa position glisse vers la position officielle (35 % de l'écart à chaque message) ;
- si le joueur **est en mouvement**, aucune correction n'est appliquée pour ne pas ralentir sa marche ;
- caché ou mort : toujours aligné sur la position officielle.

Pour les autres joueurs, l'affichage se rapproche de la cible reçue avec un lissage exponentiel (constante de temps d'environ 60 ms).

7. Le moteur de jeu

7.1 La boucle de jeu

Le moteur repose sur `requestAnimationFrame`, qui appelle la fonction `gameLoop` à chaque rafraîchissement de l'écran (en général 60 fois par seconde). Chaque image suit le même enchaînement :

1. Calcul du temps écoulé depuis l'image précédente (`deltaMs`), plafonné à 100 ms pour éviter les sauts après une mise en veille.
2. **Hôte** : `computeHostPhysics` met à jour tous les joueurs et envoie l'état aux invités. **Invité** : `guestFrameUpdate` interpole les autres joueurs et prédit son propre mouvement.
3. `drawGame` dessine la scène complète.
4. `checkVictory` teste les conditions de fin de partie.

Tous les déplacements sont multipliés par `deltaMs`, ce qui rend la vitesse du jeu **indépendante de la puissance de l'ordinateur** : un écran à 144 Hz ne fait pas courir les personnages plus vite.

7.2 La grille et les coordonnées

- Le monde est découpé en **tuiles de 16 × 16 pixels**. Une carte de 56 × 30 tuiles mesure donc 896 × 480 pixels.
- Les positions des joueurs et des meubles sont exprimées en **pixels du monde**.
- Le canvas fait 768 × 768 pixels et le monde est affiché avec un **zoom ×3** : le joueur voit donc une zone de 256 × 256 pixels du monde (16 × 16 tuiles), centrée sur lui. Le zoom donne l'effet pixel art, avec le lissage d'image désactivé.

7.3 Déplacements et collisions

Les touches pressées forment un vecteur de direction, **normalisé** pour que la diagonale ne soit pas plus rapide. La fonction `applyMovement`, commune à l'hôte et à la prédiction des invités, déplace ensuite le personnage.

- **Axe par axe** : on teste d'abord le déplacement horizontal, puis le vertical. Un joueur qui touche un mur en diagonale **glisse** le long du mur au lieu de s'arrêter net.
- **Test de collision** (`collides`) : on vérifie que les quatre coins de la boîte de 24 px ne sont pas dans une tuile de mur, que la boîte reste dans la carte, et qu'elle ne chevauche aucun meuble solide.
- **Pas de 8 px maximum** : un déplacement rapide (le dash) est découpé en petits pas, pour qu'un joueur ne puisse jamais « sauter » par-dessus un mur de 16 px.
- **Exceptions** : les cachettes ouvertes et les tapis sont traversables.

7.4 La ligne de vue et le brouillard de guerre

La fonction `hasLineOfSight` trace un segment entre deux points et le parcourt **tous les 8 pixels** : si un point tombe dans une tuile de mur, la vue est bloquée.

Pour dessiner le brouillard, chaque tuile voisine du joueur reçoit une opacité de noir :

- **0 %** (parfaitement visible) à moins de la moitié du rayon de vision ;
- un **dégradé progressif** jusqu'au rayon de vision maximal ;

- **100 %** (noir) au-delà du rayon, ou si un mur masque la tuile.

> **Optimisation —** seules les tuiles situées dans le carré du rayon de vision sont évaluées ; tout le reste de la carte est recouvert par quatre grands rectangles noirs. Sans cela, le coût du brouillard aurait grandi avec la taille de la carte et ralenti les grandes cartes.

Les tuiles de brouillard sont dessinées en 17 × 17 pixels au lieu de 16 × 16 afin de se chevaucher légèrement et d'éviter de fines lignes claires entre elles.

7.5 La caméra et l'ordre de dessin

La caméra est toujours centrée sur le joueur local. Chaque image est dessinée dans cet ordre, du fond vers l'avant :

1. sol (couleur de base) puis zones de sol colorées et leurs noms ;
2. escaliers (cartes 1 et 3) ;
3. murs ;
4. meubles, avec rotation et échelle propres à chacun ;
5. joueurs (en excluant les traqués cachés et les spectateurs) ;
6. brouillard de guerre ;
7. minimap, puis décompte noir du chasseur si nécessaire.

7.6 Animation des sprites

Chaque personnage possède une minuterie d'animation qui avance tant qu'il se déplace. Pour le chasseur, `(minuterie / 200) mod 2` choisit entre deux images de marche. À l'arrêt, une image fixe est affichée. L'orientation est choisie parmi quatre directions selon l'axe de déplacement dominant.

7.7 Les cachettes : une machine à états

Chaque meuble possède un **type** numérique. Se cacher, fouiller ou trouver un traqué se résume à **changer ce type** : par exemple `BED` devient `BED_OPEN` quand le chasseur l'ouvre, puis `BED_DEAD` s'il y trouve quelqu'un. Le rendu et les collisions lisent simplement le type actuel.

Fonction	Rôle
`isHidingSpot`	Indique si un type de meuble est une cachette.
`findInteractiveFurniture`	Trouve la cachette la plus proche à moins de 30 px de la boîte du joueur.
`toggleFurniture`	Ouvre (chasseur) ou ferme (traqué) le meuble en changeant son type.
`handleHiderAction`	Cache le traqué (téléportation au centre du meuble, mémorisation du point d'entrée) ou le fait ressortir.
`handleHunterSearch`	Ouvre le meuble ; si un traqué se trouve à moins de 10 px de son centre, il est éliminé et le meuble passe à l'état « occupé ».

7.8 Le dash

Paramètre	Valeur	Constante
Durée du dash	600 ms	`DASH_DURATION_MS`
Rechargement (après la fin du dash)	8 000 ms	`DASH_COOLDOWN_MS`
Multiplicateur de vitesse	× 2,2	`DASH_SPEED_MULT`

L'hôte détient l'état officiel du dash (`dashTimer`, `dashCooldown`). Il le déclenche sur un appui de la touche Maj **uniquement si** le traqué se déplace, n'est pas caché et n'est pas déjà en dash ou en rechargement. L'invité le **prédit** localement pour une réaction immédiate ; l'hôte reste la référence.

7.9 La minimap

La minimap est un second canvas où chaque tuile mesure 4 pixels. Elle redessine le sol, les murs et les meubles (colorés par type) puis le **point du joueur local uniquement** (rouge pour le chasseur, bleu pour un traqué). Sa taille s'adapte automatiquement aux dimensions de la carte.

7.10 Le temps et la fin de partie

- **Chronomètre** : le serveur décompte le temps de la manche et envoie `timerUpdate` chaque seconde ; l'état synchronisé le transporte aussi.
- **Compte à rebours du chasseur** : 10 secondes, décrémentées par l'hôte ; le chasseur est figé et l'écran noir tant qu'il n'est pas terminé.
- **Victoire** : chaque client évalue lui-même `checkVictory` à partir de l'état reçu (tous les traqués éliminés, ou temps écoulé avec au moins un survivant), puis affiche le message et fige l'écran.

8. Conception des cartes

8.1 Représentation

Une carte est décrite par deux structures simples :

- `isWallGrid` : un tableau à deux dimensions de booléens, vrai pour une tuile de mur. Il sert à la fois aux collisions, à la ligne de vue, au brouillard et à la minimap.
- `furnitures` : une liste d'objets `{ x, y, width, height, type, rotation, scale }`, un par meuble.

La carte n'est **jamais transmise par le réseau** : elle est reconstruite par chaque navigateur à partir de son numéro (`generateInitialState`). Seul l'état des meubles (ouvert, fermé, occupé) est synchronisé.

8.2 Les outils de construction

Outil	Rôle
`addWallBlock(gx, gy, largeur, hauteur)`	Pose un bloc de murs rectangulaire, en tuiles.
`addDiagonalWall(...)`	Trace un mur en diagonale (carte 1).
`addDoorWall(vertical, position, début, fin, portes)`	Trace un mur complet percé de portes ; chaque porte est un couple [début, longueur].
`place(type, gx, gy, largeur, hauteur, rotation, options)`	Pose un meuble à une position exprimée en tuiles, avec une taille en pixels.
`zone(gx, gy, largeur, hauteur, couleur, nom)`	Colore le sol d'une pièce et affiche son nom.
`tableSet(gx, gy)`	Pose une table entourée de ses quatre chaises.

Exemple : trois murs verticaux de la carte « Manoir », chacun percé de trois portes de 4 tuiles :

```
addDoorWall(true, 22, 1, 47, [[6, 4], [24, 4], [39, 4]]);
addDoorWall(true, 44, 1, 47, [[10, 4], [20, 4], [36, 4]]);
addDoorWall(true, 66, 1, 47, [[4, 4], [26, 4], [41, 4]]);
```

8.3 Taille et rotation des meubles

Pour tous les meubles, `width` et `height` décrivent l'**emprise réelle à l'écran**, après rotation. L'image d'origine est donc dessinée en intervertissant largeur et hauteur pour une rotation de 90° ou 270°. Cette convention permet d'utiliser la même boîte pour l'affichage et pour les collisions.

Convention d'orientation retenue : un meuble contre un mur du haut garde sa rotation de 0° ; contre un mur du bas, 180° ; un canapé tourné à 180° regarde vers le haut, face à une télévision.

8.4 Calage automatique aux murs

Sur les cartes 4 et 5, une étape finale recale certains meubles contre les murs, quelles que soient leurs coordonnées d'origine :

Meuble	Règle
Plante	Collée au mur le plus proche, quelle que soit la distance.
Canapé	Dossier collé au mur situé derrière lui s'il en est à moins de 48 px ; collé à un mur latéral à moins de 40 px.
Dressing	Toujours collé au mur du haut (à moins de 96 px) ; les dressings voisins sont alignés bord à bord.

La distance à un mur se mesure en avançant pixel par pixel depuis le centre du meuble dans chaque direction jusqu'à rencontrer une tuile de mur.

8.5 Cartes de tailles différentes

Les cartes 1 à 3 mesurent 56 × 30 tuiles. Pour les grandes cartes, une table `MAP_SIZES` associe un numéro de carte à ses dimensions ; les variables `worldCols` et `worldRows` en dérivent. Toutes les fonctions (collisions, ligne de vue, brouillard, minimap, rendu) utilisent ces variables, ce qui permet d'ajouter des cartes de n'importe quelle taille.

8.6 Vérification des cartes

Chaque grande carte a été validée par un test automatique exécuté dans le navigateur :

- les positions de départ du chasseur et des traqués ne chevauchent ni mur ni meuble ;
- un **parcours en largeur** (BFS) à pas de 8 px, utilisant la vraie fonction de collision, part du chasseur et vérifie que chaque point de départ et chaque pièce est **atteignable** ;
- un rendu en vue d'ensemble (sans brouillard) permet de contrôler visuellement la disposition.

8.7 Ajouter une nouvelle carte

1. Si la carte est plus grande que 56 × 30, ajouter son numéro et ses dimensions dans `MAP_SIZES`.
2. Ajouter une branche `else if (mapIndex === N)` dans `generateInitialState` avec les murs (`addWallBlock`, `addDoorWall`), les zones de sol (`zone`) et les meubles (`place`).
3. Définir `hunterSpawn` et `hiderSpawns`, en choisissant des points libres.
4. Ajouter l'option correspondante dans la liste déroulante de `index.html`.
5. Autoriser le nouveau numéro côté serveur dans le contrôle de `SET_MAP_` de `server.js`.
6. Tester que les points de départ sont libres et que toutes les pièces sont accessibles.

9. Interface et style

9.1 Gestion des écrans

Tous les écrans sont dans un seul fichier HTML. La fonction `showScreen` masque tous les écrans puis affiche celui demandé en retirant la classe `hidden` (`display: none`). Il n'y a donc ni rechargement de page ni routeur : le jeu est une application d'une seule page.

9.2 Identité visuelle

- **Police** : « Press Start 2P », police pixel de Google Fonts, pour tout le jeu.
- **Palette** : fond sombre (`#0d0d15`), rouge pour le chasseur (`#e63946`), bleu pour les traqués (`#2196f3`), or pour les accents (`#ffd60a`). Ces couleurs sont des variables CSS réutilisées partout.
- **Style pixel** : bordures épaisses blanches, ombres portées décalées sans flou, rendu `pixelated` des images.
- **Titre** : l'image « Cache cache titre » est affichée en fond de l'accueil (taille « contain » : toujours entière, jamais rognée) et en bannière au-dessus du salon.

9.3 Adaptation aux petits écrans

La mise en page est flexible. Sous 700 px de large, les panneaux du salon passent l'un sous l'autre. L'accueil et le salon sont centrés verticalement, tout en restant entièrement accessibles par défilement si l'écran est petit.

9.4 Le bouton de lancement

Le bouton « Lancer la partie » reflète l'état du salon : grisé et libellé « En attente d'un 2e joueur… » tant qu'il manque un joueur, actif ensuite ; pour les invités, il indique « En attente de l'hôte… ». Le serveur refuse de toute façon un lancement à moins de deux joueurs.

10. Déploiement

10.1 Le code sur GitHub

Le projet est versionné avec Git et publié dans un dépôt GitHub (privé). Le fichier `.gitignore` exclut le dossier `node_modules`, qui est reconstitué par `npm install` à partir de `package.json` et `package-lock.json`.

10.2 L'hébergement sur Render

Le jeu est déployé comme **Web Service** Render, relié au dépôt GitHub. Chaque envoi (`git push`) sur la branche `main` déclenche automatiquement une nouvelle construction et un redéploiement.

Réglage Render	Valeur
Type de service	Web Service
Langage	Node
Branche	`main`
Commande de construction (Build Command)	`npm install`
Commande de démarrage (Start Command)	`npm start` (exécute `node server.js`)
Offre	Free (gratuite)
Port	Fourni par Render dans la variable d'environnement `PORT`, lue par `server.js` (3000 par défaut en local)

10.3 Lancer le jeu en local

```
npm install
npm start
# puis ouvrir http://localhost:3000 dans le navigateur
```

10.4 Limites du plan gratuit et bonnes pratiques

- **Mise en veille** : après environ 15 minutes sans visite, le service s'endort ; le premier chargement suivant prend 30 à 60 secondes.
- **Mémoire volatile** : un redémarrage ou un redéploiement efface les salons en cours.
- **Une seule instance** : l'état des salons est en mémoire, donc plusieurs instances empêcheraient les joueurs de se retrouver dans la même partie.
- **Région** : choisir la région la plus proche des joueurs (Frankfurt pour des joueurs en France) réduit la latence.
- **CPU limité** : l'offre gratuite dispose de peu de puissance ; c'est la raison pour laquelle le trafic réseau a été fortement réduit (voir 6.6).
- **Après une mise à jour** : faire recharger la page (Ctrl+F5) à tous les joueurs pour éviter un ancien client avec un nouveau serveur.

11. Sécurité, qualité et tests

11.1 Règles contrôlées par le serveur

Même si la simulation est côté hôte, le serveur fait respecter les règles du salon :

- seul l'hôte peut lancer la partie, changer la durée ou la carte ;
- le lancement exige au moins 2 joueurs ;
- un seul chasseur et quatre traqués au maximum ;
- la carte demandée doit être un numéro valide (0 à 4) ;
- seul l'hôte peut envoyer l'état officiel du jeu ;
- impossible de rejoindre une partie en cours ou un salon inexistant.

11.2 Dépendances et vulnérabilités

Lors du déploiement, Render a signalé **7 vulnérabilités** dans les dépendances indirectes (`body-parser`, `engine.io`, `proxy-addr`, `qs`, `socket.io-parser`, `ws`, `socket.io-adapter`), dont une critique. La commande `npm audit` les a détaillées et `npm audit fix` les a toutes corrigées sans changement de version majeure : l'audit affiche désormais **0 vulnérabilité**.

11.3 Méthodes de test employées

- **Tests réseau scriptés** : des clients Socket.IO de test créent un salon, rejoignent, tentent un lancement à 1 puis 2 joueurs, mesurent la taille et la fréquence des messages.
- **Tests dans le navigateur** : exécution de scripts pour simuler des touches, mesurer les distances parcourues (marche, dash), vérifier les compteurs et l'interface.
- **Validation des cartes** : test des points de départ et parcours en largeur d'accessibilité (voir 8.6).
- **Contrôle visuel** : captures d'écran des écrans et rendu en vue d'ensemble de chaque carte.

11.4 Limites connues

- L'hôte fait autorité : un hôte malveillant pourrait modifier la partie (acceptable entre amis).
- Il n'y a pas de reconnexion en cours de partie : un joueur qui perd sa connexion quitte le salon.
- Aucun test automatisé n'est intégré au dépôt ; les tests ci-dessus sont ponctuels.
- Le code client est volumineux (un seul fichier) et mérite d'être découpé en modules.
- Deux images sont référencées mais absentes (`sol.png`, `mur.png`) : sans effet visible, le sol et les murs étant dessinés par code.
- Les écrans hérités (film, nouveau mode, chargement, game over) subsistent sans être utilisés.

12. Historique des évolutions

Les principales étapes du projet, dans l'ordre, avec l'identifiant du commit correspondant dans Git :

Commit	Évolution
`b725964`	Version initiale : import du jeu (serveur, client, 3 cartes, sprites).
`bb63171`	Correction des 7 vulnérabilités des dépendances (`npm audit fix`).
`5e9b290`	Lancement impossible avec moins de 2 joueurs (serveur et bouton grisé).
`e450eac`	Image du titre en fond de l'accueil et bannière du salon.
`a8f98eb`	Minimap : affichage du seul joueur local.
`53e6f1a`	Deux grandes cartes (Manoir 88 × 48 et Villa 104 × 56), pièces thématiques, calage des meubles aux murs, brouillard optimisé.
`78ada0e`	Réduction du lag des invités (état compact à 20 Hz, prédiction, interpolation) et dash des traqués.

13. Pistes d'amélioration

- **Serveur autoritaire** : déplacer la simulation sur le serveur supprimerait la triche et l'avantage de l'hôte, au prix d'une charge serveur plus importante.
- **Reconnexion** : permettre à un joueur déconnecté de reprendre sa place.
- **Son et musique** : ambiance, pas, grincements de portes, alerte quand le chasseur fouille tout près.
- **Nouveaux objets et pouvoirs** : lampe torche, leurres, pièges, seconde chance.
- **Plusieurs chasseurs** ou modes d'équipes.
- **Modularisation du code** : séparer `client.js` en modules (réseau, rendu, cartes, entrées).
- **Tests automatisés** : intégrer les tests de cartes et de règles à la livraison.
- **Commandes tactiles** : joystick virtuel pour jouer sur téléphone.
- **Nettoyage** : retirer les écrans hérités et les images référencées inexistantes.

Annexe A — Glossaire

Terme	Définition
Hôte	Joueur dont le navigateur calcule la simulation du salon ; le premier arrivé, avec la couronne.
Invité	Joueur qui n'est pas l'hôte : il envoie ses touches et affiche l'état reçu.
Salon	Groupe de joueurs identifié par un code à 6 caractères.
Tuile	Carré de 16 × 16 pixels, unité de la grille de la carte.
Brouillard de guerre	Zone non visible, représentée en noir autour du joueur.
Ligne de vue	Segment sans mur entre deux points, condition pour voir une tuile.
Prédiction	Calcul local anticipé du mouvement avant la confirmation de l'hôte.
Interpolation	Déplacement progressif d'un affichage vers une position cible.
Snapshot	Instantané compact de l'état de la partie envoyé aux invités.
BFS	Parcours en largeur d'un graphe, utilisé pour tester l'accessibilité des pièces.
Dash	Accélération brève des traqués, rechargée après 8 secondes.

Annexe B — Paramètres du jeu

Les valeurs ci-dessous sont celles du code actuel ; elles se modifient directement dans `client.js` et `server.js`.

Paramètre	Valeur
Taille d'une tuile	16 px
Zoom d'affichage	× 3
Vitesse des joueurs	120 px/s
Taille de la boîte de collision	24 × 24 px
Compte à rebours du chasseur	10 s
Durées de partie	120 / 180 / 240 s (120 s par défaut)
Rayon de vision	chasseur 250 px ; traqué 150 px ; zone nette = moitié du rayon
Distance d'élimination au contact	24 px
Distance d'interaction avec un meuble	30 px du bord du meuble
Distance de détection d'un traqué caché	10 px du centre du meuble
Dash	600 ms, × 2,2 ; rechargement 8 s après la fin
Fréquence d'envoi de l'état	20 par seconde (50 ms)
Rappel de l'état des meubles	toutes les 40 envois, soit environ 2 s
Battement de secours des touches	500 ms
Seuil de recalage de la prédiction	70 px
Rattrapage à l'arrêt	35 % de l'écart par message
Lissage des autres joueurs	constante de temps d'environ 60 ms
Pas maximal de déplacement	8 px
Joueurs	minimum 2 ; 1 chasseur ; 4 traqués maximum
Suppression des salons inactifs	après 1 h, vérifiée toutes les 10 min

Annexe C — Liste des types de meubles

Famille	Types
Cachettes	`BED`, `BED_DOUBLE`, `WARDROBE`, `DRESSING`, `SHOWER`, `KANGOO`, `RUG`, `BATHTUB` (et leurs états `_OPEN` / `_DEAD`)
Salon et cuisine	`COUCH`, `ARMCHAIR`, `TV`, `TABLE`, `CHAIR`, `CHAIR_TABLE`, `FRIDGE`, `STOVE`, `SINK`, `SHELF`
Bureau et chambre	`DESK`, `DESK_MAC`, `CHAIR_DESK`, `NIGHTSTAND`, `LAMP`
Autres	`TOILET`, `BOILER`, `MOTO`, `TRUNK`, `BILLARD`, `PLANT`